ADVERSARIAL EVASION OF STATIC PE MALWARE DETECTORS

Sidh Virmani

BITS GOA

2026-2027

PROBLEM STATEMENT

- Malware detectors for Portable Executables are usually trained **on static PE features.**
- These models look at things like **headers, sections, imports, byte patterns, and metadata**.
- But attackers can make small changes that keep these features and functionality the same
- These changes may not break the file’s execution but can make model classify it as benign

METHODS TO BYPASS MODELS

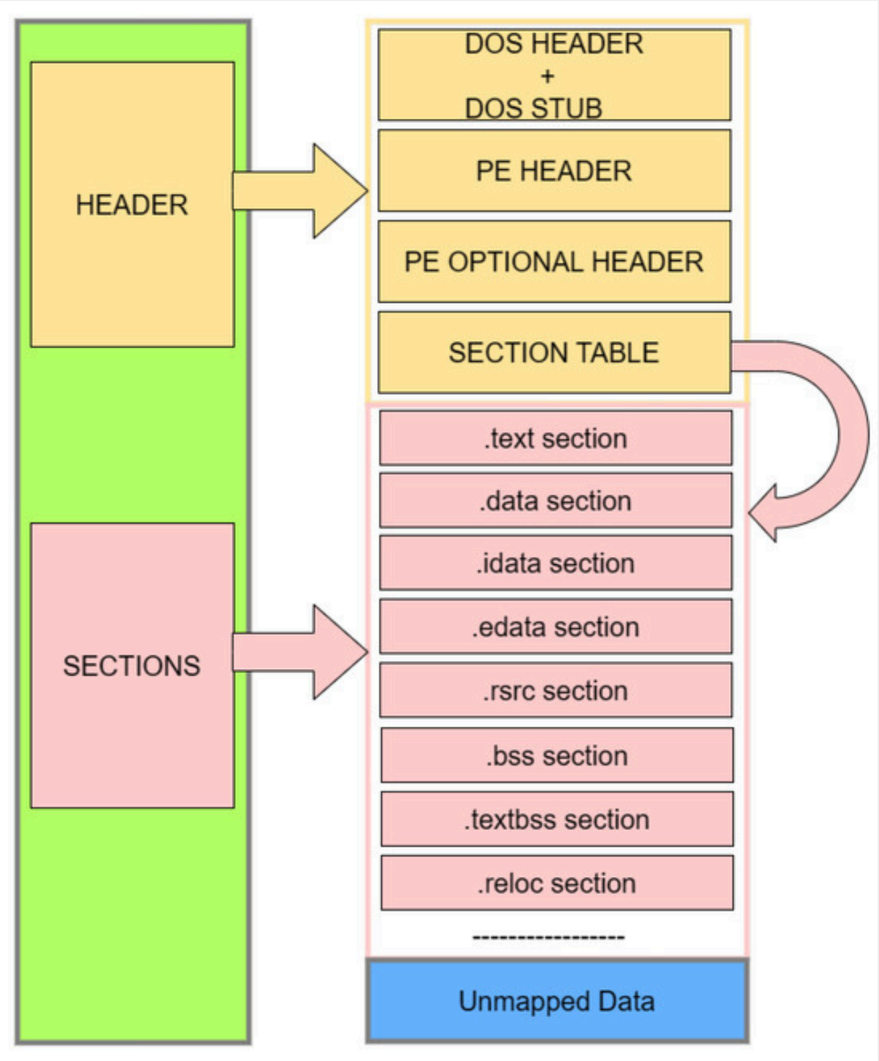
1. Benign Section Addition or Statistical Blending
2. Import Table Manipulation or API Blending
3. Header Metadata Normalization
4. Execution Path Addition
5. White/Black - box attack

STATISTICAL BLENDING

1. Malware files often have unusual byte patterns (also called high entropy values)
2. An .exe file is internally structured into sections. Eg. code section, data section, text section etc. The attacker can create a new section full of “benign looking data and bytes”
3. Since the overall executable now has low entropy and not a lot of unusual byte patterns, the model might mislabel it as benign
4. Hence, this method makes malware look “statistically” similar to a normal file

Citations:

1. Evaluating Realistic Adversarial Attacks against Machine Learning - https://www.mdpi.com/1999-5903/16/5/168
2. Learning to Evade Static PE Machine Learning Malware - https://arxiv.org/pdf/1801.08917



The PE file is the standard binary format of DLLs and Executables in the Microsoft Windows family. Its layout contains a Header area, a Section area and the Unmapped Information area. As shown in the figure, the Header area contains the DOS Header, DOS Stub, PE Header, PE Optional Header and Section Table. Both the DOS Header and DOS Stub are used for compatibility with the MS-DOS system. **The PE Header contains global information about the PE file.**

The statistical blending (section addition) method involves attacker adding a section full of benign looking data to mislead the model

IMPORT TABLE MANIPULATION

1. API's in a PE file are the functions provided by Windows/system libraries
2. CreateFile() → open file, ReadFile() → read file, VirtualAlloc() → allocate memory are examples of some API's belonging to a benign PE file
3. These API's are stored in the import table. The import table is present inside the PE Optional Header pointing to data stored in .idata section
4. If the attacker adds a malware API and floods the table with lots of normal API's, the model may get bypassed.

Citations:

1. Evaluating Realistic Adversarial Attacks against Machine Learning - https://www.mdpi.com/1999-5903/16/5/168
2. Learning to Evade Static PE Machine Learning Malware - https://arxiv.org/pdf/1801.08917

Example

IMPORT TABLE

Before manipulation:

VirtualAlloc
WriteProcessMemory ← suspicious

After manipulation:

VirtualAlloc
WriteProcessMemory ← suspicious
printf
GetSystemTime
MessageBox
LoadString

HEADER METADATA NORMALIZATION

1. Metadata of a PE file is the extra info about the file, not the actual code
2. The most usual place of storing malware code is the .text section.
3. A benign PE headers contain metadata like timestamp, OS version, checksum
4. Malware often has fake timestamps, missing fields, unusual values.
5. The attacker can simply change the metadata of the file to make it look like a completely normally compiling program

Citations:

1. Static Analysis and Machine Learning-based Malware Detection System using PE Header Feature Values - Semantic Scholar - https://pdfs.semanticscholar.org
2. Evading Static Machine Learning Malware Detection Models – Part 2: The Gray-Box Approach - https://blog.compass-security.com/2020/11/evading-static-machine-learning-mal ware-detection-models-part-2-the-gray-box-approach/

EXECUTION PATH ADDITION

1. Model sees file as data, OS executes specific paths
2. So, if we bring some simple change in control flow of the program such that OS executes malicious path (an if-else condition is the simplest example)
3. The model looks at numbers, patterns and statistics instead of logic, intent and the control flow so if the code of execution path is written in a deceiving enough way, not effecting the overall feature distribution a lot, attack may bypass the model

Citations:

1. Explainability Guided Adversarial Evasion Attacks on Malware Detectors - https://arxiv.org/html/2405.01728v1
2. Learning to Evade Static PE Machine Learning Malware - https://arxiv.org/pdf/1801.08917

RL - BASED EVASION

1. The core idea of this approach is to let an AI automatically learn how to modify malware step-by-step until the model says “benign”
2. We train a reinforcement model to make valid changes like add section, modify metadata, change imports etc until it learns what bypasses the model and what doesn’t
3. This method requires some access to model’s scores or results on files given to it
4. This is actually called a white/black-box attack which is best implemented using reinforced learning

Citations:

1. Learning to Evade Static PE Machine Learning Malware - https://arxiv.org/pdf/1801.08917

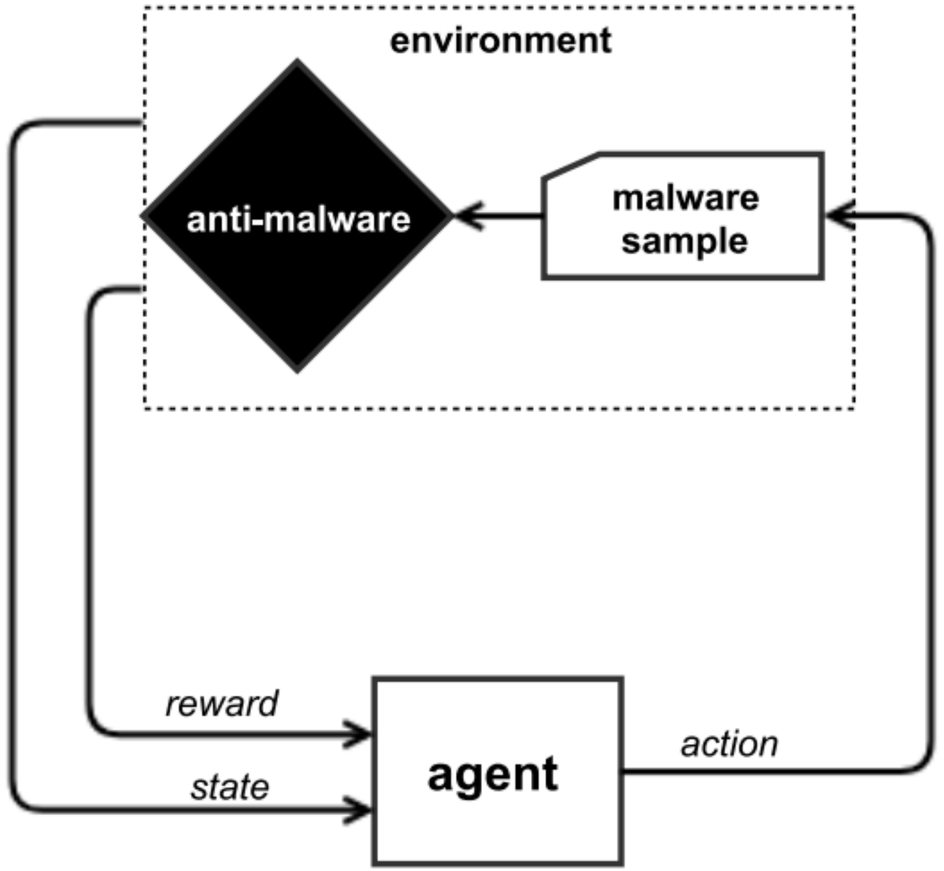


Figure 1: Markov decision process formulation of the malware evasion reinforcement learning problem.

(2) *White-box attacks against models that report a maliciousness score.* The attacker has no knowledge about the model structure, but has unlimited access to probe the model and may be able to use heuristics that take advantage of the revealed scores to search for evasive variants. Virtually every machine learning model can produce a score given a query, but deployed models often choose a threshold on that score to determine malicious / benign.

(3) *Binary black-box attacks.* The attacker has no knowledge about the model, but has unlimited access to probe the model. The model output is a single bit indicating whether the sample is considered benign or malicious.

Figures from: Learning to Evade Static PE Machine Learning Malware - https://arxiv.org/pdf/1801.08917

THANK YOU

Sidh Virmani
BITS Goa
2026-2027